



Agenda

- ① Revise Data Warehouse
- ② Fact table vs Dimension table
- ③ Realistic OLTP → OLAP Dimension Modelling
- ④ Star Schema
- ⑤ Snowflake Schema
- ⑥ Galaxy Schema
- ⑦ Slow Changing Dimensions
- ⑧ Surrogate Keys

① Hive as Data Warehouse

② Architecture of Hive

Introduction to data warehousing

Storage unit to support business analytics and reporting.

Typically a data warehouse contains a large volume of current and historical data.

Dimension Modelling

① What are the top 10 books sold weekly? For last 10 years

Fact Table

The numbers or metrics that we want to measure

Dimension Table

The when, who, what, where in business questions for which we need to measure results

Above scenario

top 10 sold books

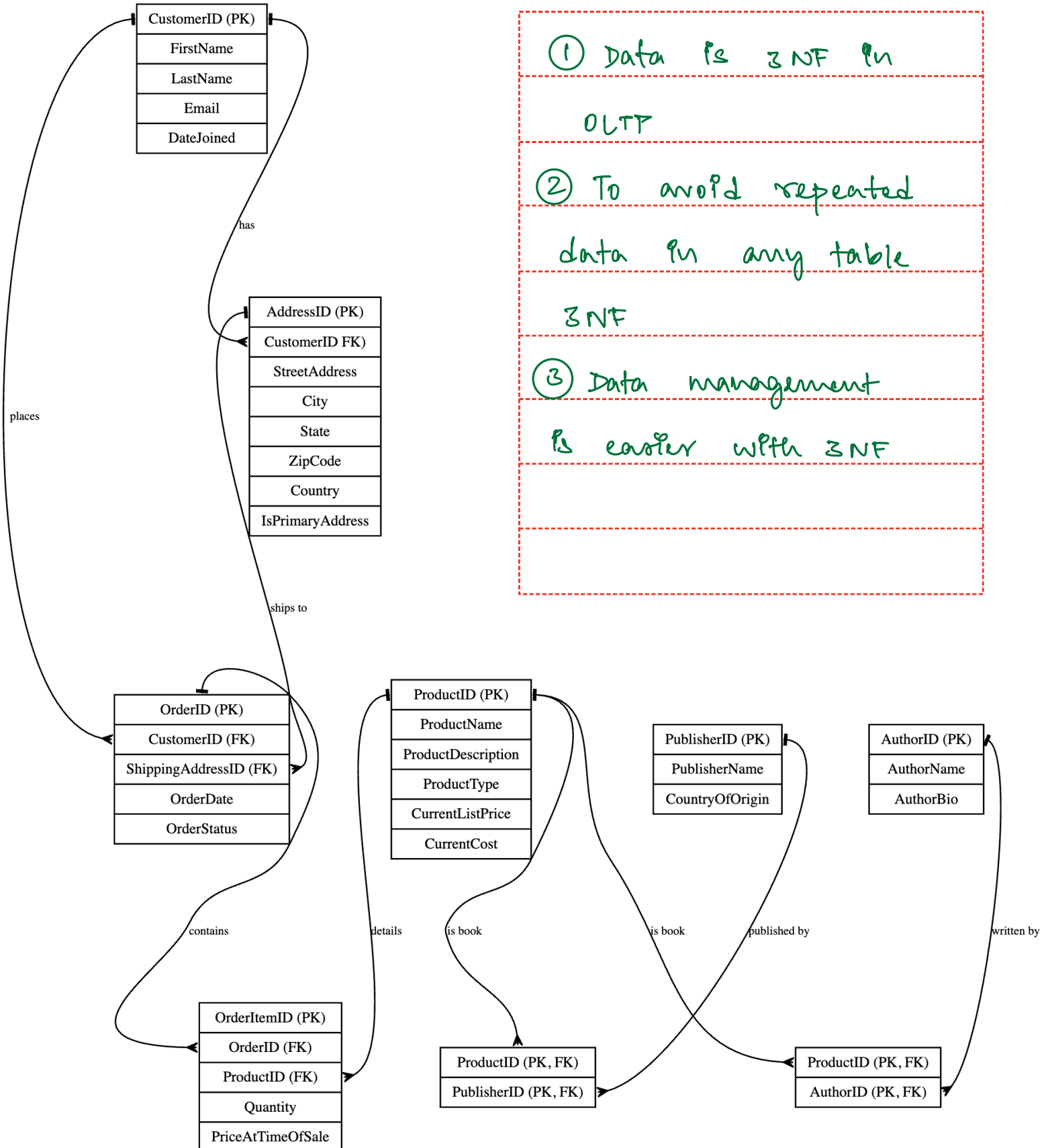
who → customer

when → Date/time

what → products

where → Geographic data

OLTP system — In Relational Database



OLAP System

When?

Dim - Date

Full Date

Day of Week

Week of Year

Month

Quarter

Year

Date ID

Who?

Dim - Customer

Customer ID

Full Name

Email

Phone Number

Date Joined

age

Customer ID

What?

Dim - Product

Product Name

Product Desc

Product Type

Genre

Publisher

product ID

Where?

Dim - Geography

Street

City

State

Zipcode

Country

Geo ID

Fact Table → Per Customer, per product, what is Qty sold,
and Revenue

Foreign Key for Date, Customer, Product & Geography

* Quantity sold

* Revenue

* Profits

Amazon

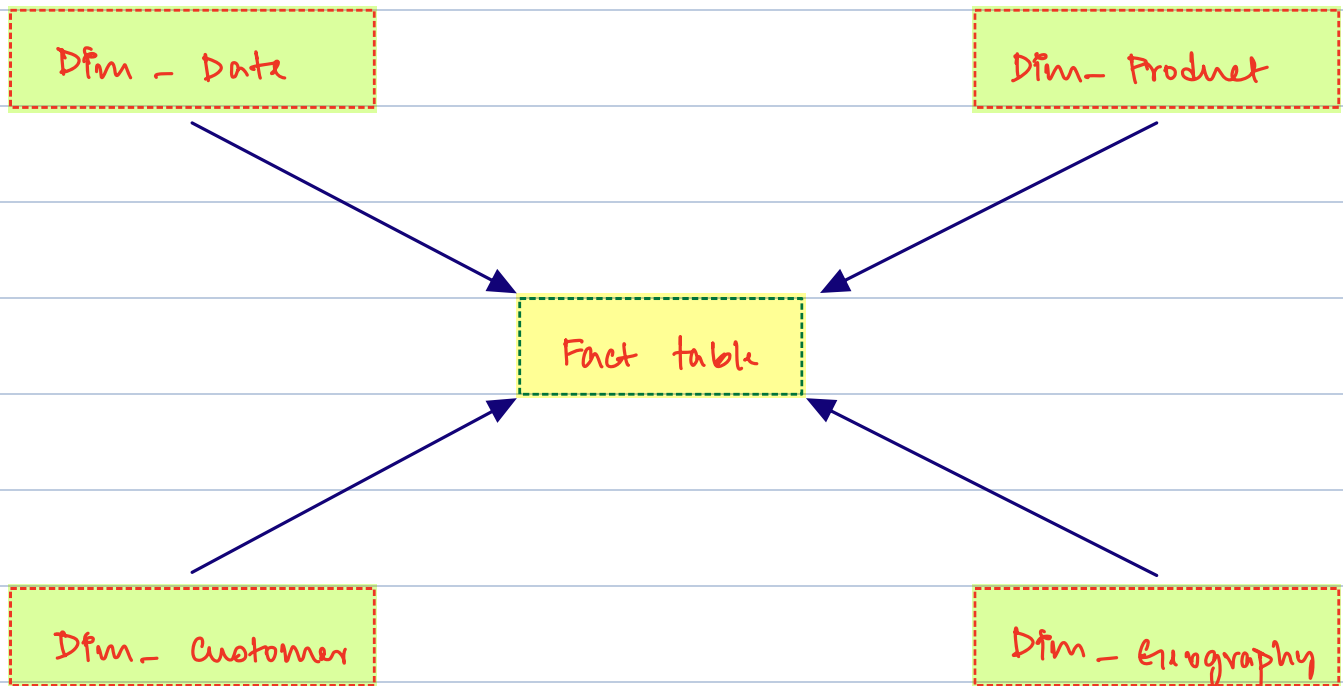
Primary Key :

1 Design Data Intern

Foreign Key :

2 Peter Thiel

Star Schema → 90% we prefer Star Schema for reporting



Structure:

Fact table at the center and dimension tables at edges joined using PK (Dim) and FK (Fact)

Pros:

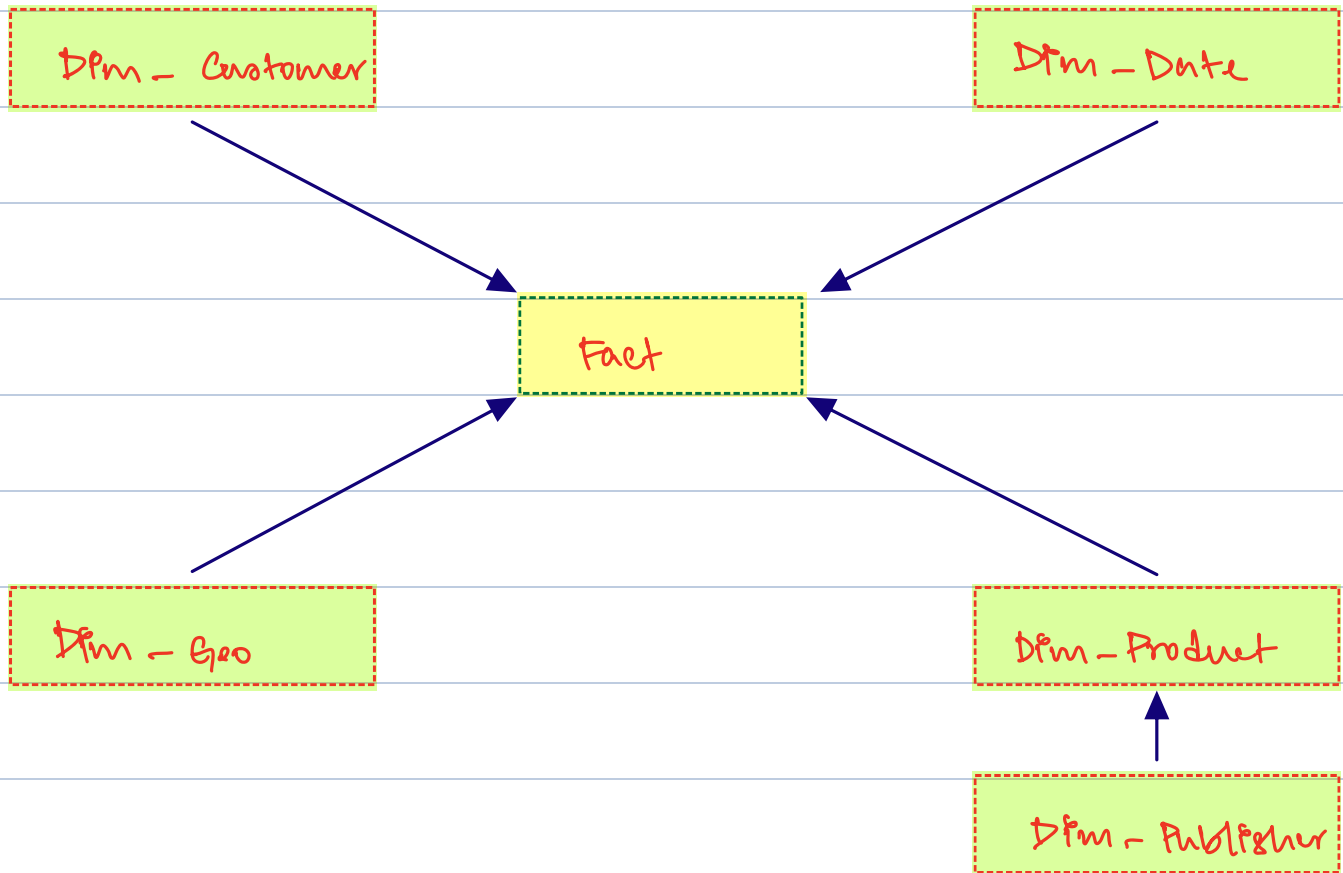
One of the simplest schemas

One of the most powerful schema

Cons:

Extreme Duplication of Data

Snowflake Schema



Structure:

Fact table at center and Dimensions at Edges.

Some Dimensions have one level of normalization

Advantages

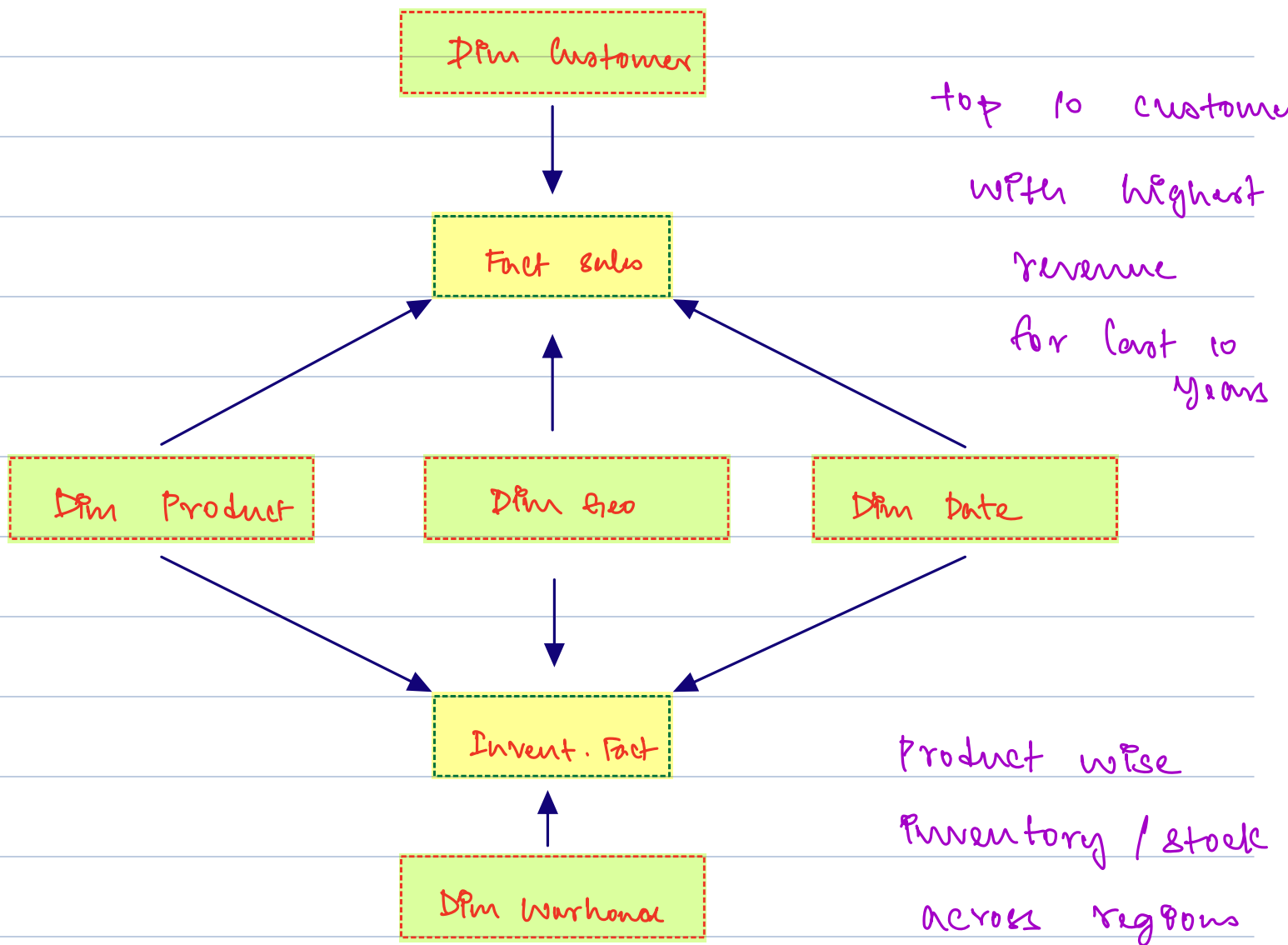
- ① Better optimization of storage

Disadvantages

- ① Because of volume of data the join with 2 Dimensions is an additional cost

Galaxy Schema

→ Imagine if you want to track inventory along with the sales of Book



Multiple DIMs that are common between 2 Fact tables → Galaxy schema

Cons

ETL becomes unnecessarily complex

Slowly changing Dimensions

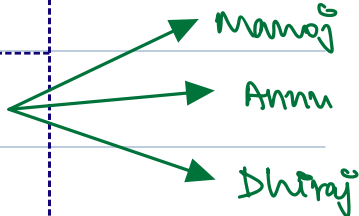
Use Case:

Imagine you have a book "ABCDE" published on "Dhiraj's Substack"

You later change your publisher to "O'Reilly Media"

How will you make sure this is addressed?

P-Key	Product Name	Genre	Publisher
101	ABCDE	Horror	Dhiraj Sub



↪ primary key

Slowly Changing Dimension - Type 0

I will never change my Dimension even if it changes in my source

P-Key	Product Name	Genre	Publisher
101	ABCDE	Horror	Dhiraj Sub

Slowly Changing Dimension - Type 1

I will keep the latest change as part of my Data Warehouse

P-Key	Product Name	Genre	Publisher
101	ABCDE	Horror	Ditrag Sub O'Reilly

Primary Key

What is the logical fix for this issue?

Keep each change in source recorded as individual entry in my Data Warehouse

Problem

Cannot have Duplicate Primary Keys

Surrogate Keys

The unique Identifier for each state in your Dimension tables

Slowly changing Dimension - Type 2 → with surrogate key

P-Key	Product Name	Genre	Publisher	Start Dt	End Dt	curr
1	101	Horror	Shiraj	13/jul	14/jul	False
2	101	Horror	O'reilly	14/jul	NA	True

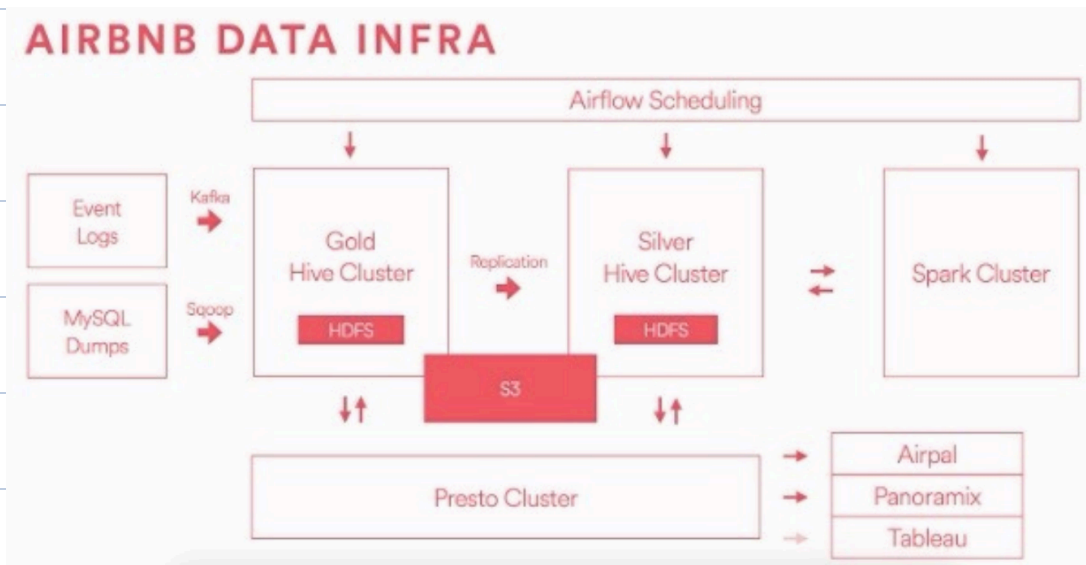
Surrogate key

Maintaining fixed set of changes

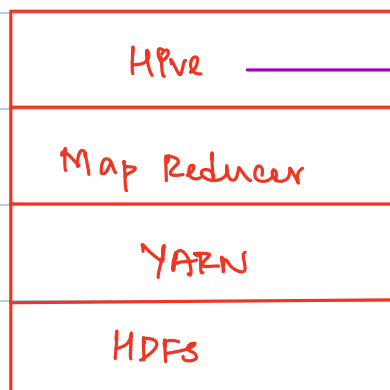
→ we have multiple ways of handling SCD. However in majority of production we use Type 2

- ① Dimension modelling depends on use case
 - ② Fact and Dimension tables
 - ③ Schemas
 - ④ Slowly changing Dimension modelling
- Golden Data

Hive as Data Warehouse



Data warehousing solution Hive

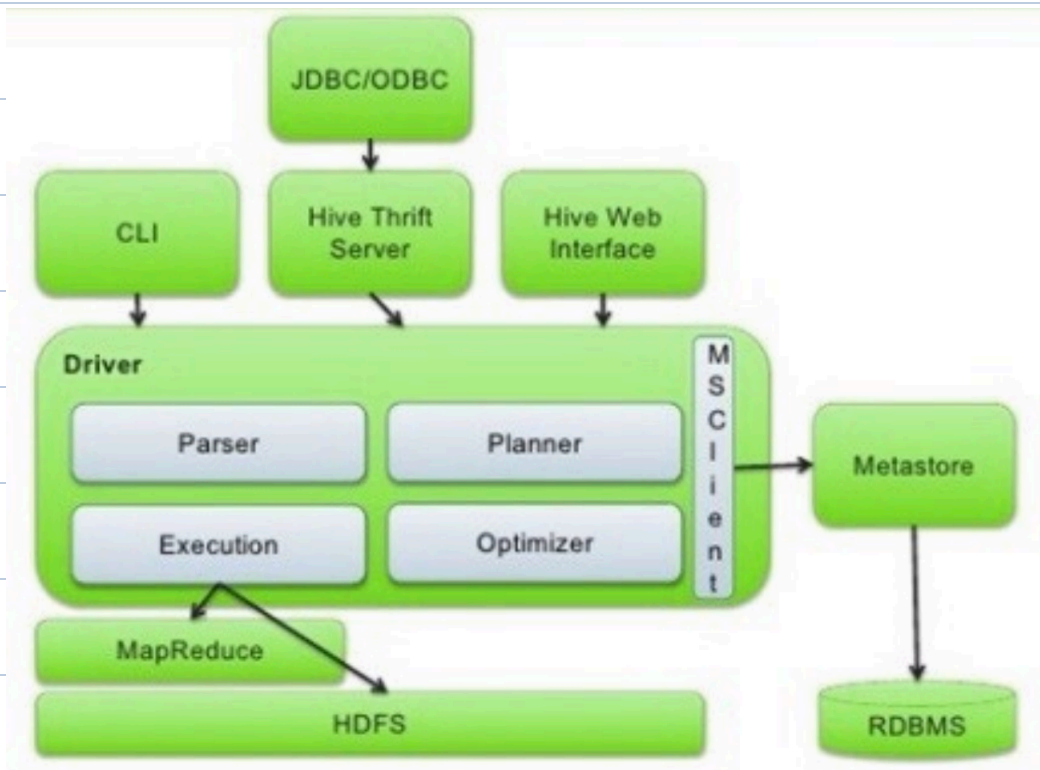


one layer on top of your execution engine

- ① DB is stored as Folder
- ② table is stored as Folder

Hive → Has a client server Architecture

the user can write SQL as input and it will process data from Distributed Data Source



Components

client:

Beeline then

client which

communicates

with your

server using

JDBC connection

Next

- ① components of Hive
- ② Demo of Hive
- ③ Optimization